Setup — data repository, token and Notion polling

Everything the app needs on the GitHub side, and how Notion reads the result. The same material, laid out for printing and handing to whoever built the Notion integration, is in **Setup-data-repository.pdf** in the repository. Do this once per season, not once per flight. Twenty minutes, no command line.

The app itself is documented separately in the manual.

---

1. Why a second repository

The app writes the flight table straight into a repository through the GitHub Contents API. Every entry is two commits — one for the JSON, one for the CSV.

If those commits land in the repository that GitHub Pages builds from, **every entry triggers a Pages rebuild**. Pages throttles at roughly ten builds an hour, so a flight with a hundred entries would jam the queue and could delay the app itself from loading. Keep the app and the data apart:

Repository	Contains	Visibility
`<owner>/gb_notion_frontend`	the app, served by GitHub Pages	public — Pages needs it, unless you have Pro
`<owner>/gb_flight_data`	the flight tables	your choice, see below

Public or private for the data? Public means the position, track and ballast history of every flight are readable by anyone who finds the URL. Private keeps them behind the token. Notion can poll a private repository perfectly well through the API — only the `raw.githubusercontent.com` shortcut stops working. Unless you want the flights to be openly followable, make it private.

2. Create the data repository

1. github.com → **New repository**
2. Name: `gb_flight_data`
3. Private, unless you decided otherwise
4. **Tick "Add a README file"** — this matters. A repository with no commits has no branch yet,

and the Contents API cannot write into a branch that does not exist. Without this the first entry fails with a 404 and the cause is not obvious.

5. Create repository

Nothing else. The app creates the `data/` folder and the files inside it on its own.

3. Create the token

Fine-grained tokens are scoped to single repositories, which is what you want here: a token that can write flight logs and nothing else.

Go straight to the page. GitHub has been renaming this corner of the settings — where the documentation still says *Developer settings*, newer accounts show *Credentials* — but the address has not moved:

```
https://github.com/settings/personal-access-tokens
```

Then **Generate new token**. Or open a pre-filled link, which sets the name and the permission and leaves only the repository to pick:

```
https://github.com/settings/personal-access-tokens/new?name=basket-write-2026&expires_in=180&contents=write
```

4. **Token name**: something you will recognise later, e.g. `basket-reporting-2026`
5. **Expiration**: set it to shortly after the season, not "no expiration". A token that expires is a token you cannot forget about.
6. **Repository access**: *Only select repositories* → pick `gb_flight_data` alone
7. **Permissions** → **Repository permissions** → **Contents** → **Read and write**

There is no single "read & write" switch on a fine-grained token — that is the classic token with its coarse `repo` scope. Here you pick one named permission at a time and give each its own level. Under *Permissions* there are three groups: *Repository permissions*, *Organization permissions* and *Account permissions*; you want the first. In the newer interface the list starts empty and you click **Add permissions** and search for the one you need, in the older one every permission is listed with a dropdown beside it.

The list is alphabetical, so it opens on **Actions**. That is not the one — keep going down to **Contents**, three rows further.

| Permission | Level | Note | |---|---|---| | Actions | No access | The first row in the list, and the wrong one. A token with *Actions: Read and write* can start and stop workflow runs and still cannot write a log row; the app answers with a 403. | | **Contents** | **Read and write** | The one you need. It covers creating, reading, updating and deleting files, which is exactly what the app does through the Contents API. | | Metadata | Read-only | Added automatically as soon as any other permission is set, cannot be switched off, and grants nothing beyond the repository's name and description. | | everything else | No access | Not Administration, not Actions, not Workflows. Contents alone is enough. |

8. **Generate token**, then copy it. GitHub shows it once. If you lose it, revoke it and make a new one — there is no way to read it back.

The token looks like `github_pat_` followed by about eighty characters.

Where the token ends up

On each device, in the browser's local storage, unencrypted. That is a deliberate trade: no server, no account, no login, and the app works with no signal. The consequences:

– If an iPad is lost, **revoke the token on GitHub**. That is the whole remedy, and it takes fifteen seconds. Everything already written stays.

– Anyone with physical access to an unlocked device can read it out of the browser storage.

– The token cannot touch anything but the one data repository, so the damage ceiling is: flight logs written or deleted.

If that is not acceptable, put a Cloudflare Worker in front of the GitHub API holding the token server-side, and point `API` in `index.html` at the Worker. The app never needs to know the token in that arrangement.

4. A second token, for viewers

Only if you want to let somebody outside the crew read the archive — a club, an organiser, whoever drove the retrieve. Skip it otherwise; everything else works without.

A viewer code has to carry a token, because the repository is private and there is no reading it without one. It must not be the token above: that one can write, and a token on somebody else's device is a token they can take out of it. So a viewer code carries a second one, which GitHub itself will not let write.

Same page, **Generate new token** again — and it has to be a **fine-grained** token, the kind whose value begins `github_pat_`. The other kind, a classic token beginning `ghp_`, cannot be narrowed down to reading: the only scope that opens a private repository to it is `repo`, and that scope writes as well. A classic token in this field would read beautifully and write just as beautifully, which is the one thing it must not do.

1. **Token name**: `basket-viewer-readonly`, or anything you will recognise
2. **Expiration**: a year is reasonable. When it expires the viewers see nothing until you put a new one in; nothing else is affected.
3. **Repository access**: *Only select repositories* → `gb_flight_data` alone
4. **Permissions** → **Repository permissions** → **Contents** → **Read-only**

This is the step that matters. If it says *Read and write*, the whole exercise is pointless. Check it on the summary page before you press Generate: it must read *Contents: Read*. GitHub adds *Metadata: Read-only* by itself — that is correct and cannot be turned off.

5. **Generate token**, and copy it — GitHub shows it once.

In the app it goes into *Settings* → *GitHub connection* → *Read-only token*, under the one you entered above. Then *Save settings*. The *Viewer code* button stays dead until it is there, and says why.

Before you hand the first code out, press **Check read-only token** beside it. The app offers GitHub a write that cannot possibly go through and reports whether it was refused — which is the only honest way to ask, since GitHub will not say what a token may do. If it comes back *this token CAN write*, the token is the wrong one: either *Contents* was left on *Read and write*, or it is a classic token.

What a viewer code carries: the repository, this read-only token, and the flights you chose. Not the write token, not the master password, not the WhatsApp numbers, not the map key.

5. Enter it in the app

On the device that will be the **master** — normally the pilot in command's iPad:

1. Menu → **Settings** → *Unlock settings*, password `1234`
2. **Setup role**: Master
3. **Flight**: flight ID, callsign, both pilots
4. **Ballast**: weight per bag, full ready ballast, quick drop amounts
5. Open **GitHub connection**: `<owner>/gb_flight_data`, branch `main`, folder `data`, paste the token — and, if you made the read-only one in section 4, that too
6. **Check connection** — it should name the repository back to you
7. **Save settings**

The other devices do not need any of this typed again — see section 7.

6. First flight, and what appears

Set a flight ID, post one entry. Within a few seconds the data repository contains:

```
data/<flight-id>.json          the table, every row, sorted by time
data/<flight-id>.csv           the same rows as CSV
data/<flight-id>.deleted.json  ids that were deleted, so they stay deleted everywhere
data/_setup.json               the shared setup the followers read
```

The file names come from the flight ID with anything that is not a letter, digit, hyphen or underscore replaced by a hyphen. `GB 2026/01` becomes `GB-2026-01.json`.

No workflow, no Action, no build step. The app is the only writer.

7. Adding the other crew members

How a device becomes the master: it knows the master password, **5678**. On its first start every device is asked *Is this the master device?* — the password makes it the master, *Join as follower* makes it a follower, and a device set up through an invitation link is a follower without being asked. There is no default and nothing is assumed; the question comes back on each start until it is answered.

The settings password **1234** is a different thing entirely: it opens the settings screen on any device, master or not, so anyone can correct a reporter name or read the setup. Only switching a device *to* master asks for 5678.

One device is the **master** and owns the flight setup. The others are **followers**.

To bring a device on board: on the master, Menu → **Share setup with another device**. A link appears. Send it to the other device however you normally send things — Signal, WhatsApp, AirDrop, a message to yourself. Opening it in the browser there shows what is about to be applied and asks for a yes. On yes, that device takes on the whole setup and marks itself a Follower.

The token in the link. The checkbox in the share sheet decides whether the token travels with it. With the token, the other device can send immediately and nothing else has to be typed. Without it, someone has to enter the token there by hand, and until then that device records but cannot send. Send a link containing the token over a channel you would send a password over, and delete the message afterwards.

What the link does not carry: the reporter name, whether the device is personal or shared, the colour scheme and the confirmation tone. Those belong to the device and the person holding it, and are left alone.

Staying in step. Every time the master saves settings, it publishes them to `data/_setup.json`. Followers pick that up within about thirty seconds and apply it, with a short note on screen. Change the flight ID, rename a pilot, add a WhatsApp recipient or edit a message template on the master, and the whole crew has it without anyone touching a setup screen. A follower can still be unlocked and edited locally, but the next publication from the master wins.

Two masters would fight over `_setup.json` and each undo the other. Exactly one device is the master.

8. Polling from Notion

The endpoint to use, private or public repository alike:

```
GET https://api.github.com/repos/<owner>/gb_flight_data/contents/data/<flight-id>.json
Authorization: Bearer <token>
Accept: application/vnd.github.raw
X-GitHub-Api-Version: 2022-11-28
```

`Accept: application/vnd.github.raw` returns the file itself rather than a JSON envelope with base64 inside it. Without that header you get the envelope and have to decode `content` yourself.

Rate limit: 5000 requests per hour per token, shared with whatever the iPads are doing. A thirty-second poll is 120 requests an hour — comfortable. If you send an `If-None-Match` header with the ETag from the previous

response, unchanged files answer `304` and cost nothing at all.

Use a separate token for Notion with the same repository and *Contents: Read* only. Notion never needs to write, and a read-only token in a third-party integration is a much smaller thing to lose.

Import rule: upsert on `id`. The file always holds the complete set of rows, not a delta — appending on every poll would multiply the table. Poll `<flight-id>.deleted.json` alongside and archive or hide the rows whose ids appear there.

The alternative, for public repositories only:

```
GET https://raw.githubusercontent.com/<owner>/gb_flight_data/main/data/<flight-id>.csv
```

No token needed, but it sits behind a CDN with roughly a five-minute cache. Fine for working through a flight afterwards, too slow to follow one live.

9. Checklist before a flight

– [ ] Token not expired — check the date on GitHub
– [ ] Master shows *Check connection* naming the data repository
– [ ] Flight ID set, and different from the last flight
– [ ] Every device shows the same flight in its header
– [ ] Exactly one device is set to Master
– [ ] A test entry appears in the repository, and in Notion
– [ ] Test entry deleted afterwards, so the flight starts empty
– [ ] Start ballast entered, or the first inventory planned for before launch

10. When something does not arrive

The header says `no repository` — repository or token missing in the settings on that device.

Entries stay `queued` — no connection, or the token has expired or been revoked. *Send now* in the menu shows the actual error rather than failing quietly.

A `404` on the first entry — the data repository has no branch yet. Add a README to it through the GitHub web interface and try again.

A `403` — the token lacks *Contents: Read and write*, or does not include this repository.

Notion shows duplicates — it is appending instead of upserting on `id`.

Notion is minutes behind — it is polling `raw.githubusercontent.com` instead of the API.

A follower will not take the setup — it is set to Master. One master only.